

Nova Luxury Furnishings Presentation Documentation

This document is a slide-by-slide reference for PowerPoint presentations on the Nova Luxury Furnishings project - each section is documented consistently and completely.

Document Structure

Each section follows the same format for consistency:

1. **What it is** - Clear explanation of the concept
 2. **Where it lives in the code** - Direct file paths and specific locations
 3. **Why it matters** - Business and technical value
 4. **Code example** - Working code snippet
-

1. Title Slide

What it is: A full-stack e-commerce application for luxury furniture shopping, built with Node.js backend and React/TypeScript frontend.

Where it lives in the code:

- Project overview: [README.md](#), [package.json](#)
- Backend entrypoint: [backend/index.js](#)
- Route bootstrap: [backend/src/modules/index.routes.js](#)

Why it matters: Sets context for the audience; shows what will be demonstrated and the tech stack used.

2. REST API

What it is: The REST API exposes endpoints for resources (products, users, orders) using standard HTTP verbs for predictable, resource-oriented interactions.

Where it lives in the code:

- Route modules: [backend/src/modules//routes.js](#)
- Product routes example: [backend/src/modules/product/product.routes.js](#)

Why it matters: Organizes backend behavior around resources so frontend developers and tools can interact predictably with the API.

Code example:

```
productRouter.route('/').get(cacheResponse(60*1000), getAllProduct).post(protectedRoutes, addProduct)
```

3. HTTP Methods

What it is: HTTP methods describe the action a request intends: GET (read), POST (create), PUT (update), DELETE (remove).

Where it lives in the code:

- Defined per route in: [backend/src/modules/product/product.routes.js](#)

Why it matters: Using the correct method makes APIs intuitive and helps tools and caches behave correctly.

Code example:

```
productRouter.route('/:id')
  .get(getSingleProduct)
  .put(updateProduct)
  .delete(deleteProduct)
```

4. JSON

What it is: JSON is the data format used for requests and responses between frontend and backend - human-readable and language-agnostic.

Where it lives in the code:

In `backend/index.js` - body parser setup:

```
import express from 'express'
app.use(express.json())
app.use(cors())
app.use(morgan('dev'))
```

In `frontend/src/lib/api.ts` - Axios client configuration:

Why it matters: JSON makes frontend-backend communication simple, consistent, and easy for tools to parse and validate.

Code examples:

Frontend Axios client setup:

```
export const api = axios.create({
  baseURL: apiBaseUrl,
  timeout: 20_000,
  headers: { "Content-Type": "application/json" },
})
```

Signup request (POST `/api/v1/auth/signup`):

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "secret123"
}
```

Add-to-cart request (POST `/api/v1/cart`):

```
{
  "productId": "64a1f2e5b3c7d9a0f1e2d3c4",
  "quantity": 1
}
```

Success response:

```
{
  "status": "success",
  "cart": {
    "items": [{ "product": "64a1f2...", "quantity": 2 }],
    "total": 1799.98
  }
}
```

5. Client-Server Architecture

What it is: Describes how the frontend (client) and backend (server) interact: user actions on the client become API requests handled by the server.

Where it lives in the code:

In `backend/index.js` - server bootstrap:

```
import express from 'express'
import { dbConnection } from '../database/dbConnection.js'
import { bootstrap } from '../src/modules/index.routes.js'
import cors from 'cors'
import morgan from 'morgan'

const app = express()
app.use(cors())
app.use(express.json())
app.use('/uploads', express.static('uploads'))
app.use(morgan('dev'))

bootstrap(app)
dbConnection()

app.listen(port, () => {
  console.log(`Backend API listening on http://localhost:${port}`)
})
```

In `frontend/src/lib/api.ts` - frontend API client:

Why it matters: Keeps responsibilities separated (UI logic in client, data and business rules on server). Improves security, testability, and scalability.

Add-to-cart flow example:

Frontend request:

```
api.post('/api/v1/cart', {
  productId: '64a1f2e5b3c7d9a0f1e2d3c4',
  quantity: 1
})
```

Backend handler:

```
// backend/src/modules/cart/cart.controller.js
export const addToCart = async (req, res) => {
  // validate input
  // update DB
  // return JSON response
}
```

6. API Consumption

What it is: How the frontend calls the backend via a shared HTTP client (Axios) with automatic auth token attachment.

Where it lives in the code:

- Shared Axios client: [frontend/src/lib/api.ts](#)
- Request interceptor: same file

Why it matters: Centralizes request configuration and auth handling so components don't repeat this code.

Code example:

```
api.interceptors.request.use((config) => {
  const token = typeof window !== 'undefined' &&
    window.localStorage.getItem(TOKEN_KEY)
  if (token) {
    (config.headers as Record<string, string>).token = token
  }
  return config
})
```

7. Authentication

What it is: JWT-based authentication with email verification and protected routes for securing sensitive endpoints.

Where it lives in the code:

In `backend/src/modules/auth/auth.controller.js` :

```
const issueAuthToken = (user) =>
  jwt.sign(
```

```
{ userId: user._id, role: user.role },  
process.env.JWT_KEY  
)
```

In `backend/src/modules/auth/auth.routes.js` - protected route definitions

In `backend/src/middleware/` - protected route middleware

Why it matters: Ensures only authenticated users can access protected endpoints and supports account recovery and verification flows.

Code example:

```
const issueAuthToken = (user) =>  
  jwt.sign(  
    { userId: user._id, role: user.role },  
    process.env.JWT_KEY  
  )
```

8. Database Integration

What it is: MongoDB with Mongoose to store and validate data (users, products, orders, carts) with schema-level validation.

Where it lives in the code:

In `backend/database/dbConnection.js` :

```
import mongoose from "mongoose"  
  
export const dbConnection = () => {  
  const mongoUri = process.env.MONGODB_URI || process.env.DB_STRING ||  
    'mongodb://localhost:27017/halim'  
  
  mongoose.connect(mongoUri)  
    .then(() => console.log("db is connected successfully"))  
    .catch((err) => console.log('db failed', err))  
}
```

In `backend/database/models/` all data models like `product.model.js` , `user.model.js` , etc.

Why it matters: Stores persistent data, enforces schema-level validation, and supports queries for all application features.

Code example:

```
mongoose.connect(mongoUri)  
  .then(() => console.log('Database connected'))  
  .catch(err => console.error('Connection failed:', err))
```

9. Swagger Documentation

What it is: Swagger (OpenAPI) provides interactive, browser-based documentation and a testing UI for the API.

Where it lives in the code:

- Configuration: [backend/swagger.config.js](#)
- Mounted at: [backend/src/modules/index.routes.js](#) under `/api-docs`

Why it matters: Developers and testers can explore and test endpoints without writing client code; improves discoverability.

Code example:

```
import swaggerUi from 'swagger-ui-express'

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs))
```

10. File Uploads

What it is: Support for uploading image files (product images) from client to server using multipart/form-data.

Where it lives in the code:

- Upload helper: [backend/src/services/fileUpload/fileUpload.js](#)
- Product upload routes: [backend/src/modules/product/product.routes.js](#)
- Static serve: [backend/index.js](#) (`app.use('/uploads', express.static('uploads'))`)

Why it matters: Allows product images to be stored and served to users; important for product listings and visual presentation.

Code example:

```
uploadFields([
  { name: 'imgCover', maxCount: 1 },
  { name: 'images', maxCount: 10 }
])
```

11. Email Service

What it is: Service to send verification and notification emails to users (signup verification OTP, password reset, etc.).

Where it lives in the code:

- Email sender: [backend/src/services/email/sendEmail.js](#)
- Templates: [backend/src/services/email/verificationEmailTemplate.js](#)
- Triggered from: auth controller during signup flows

Why it matters: Email verification prevents fake accounts and improves security; emails are needed for password recovery and user communication.

Code example:

```
await transporter.sendMail({
  from: senderEmail,
  to: userEmail,
  subject: 'Verify Your Account',
  html: verificationEmailTemplate({ name, otp })
})
```

12. Error Handling

What it is: Centralized error handling and validation mechanism to format and respond to errors consistently.

Where it lives in the code:

- Global error middleware: [backend/src/middleware/globalError.js](#)
- Validation helpers: [backend/src/middleware/validation.js](#)

Why it matters: Prevents leaking internal errors to clients, makes debugging easier in development, and keeps responses consistent in production.

Code example:

```
function globalError(err, req, res, next) {
  res.status(err.statusCode || 500).json({
    error: err.message
  })
}
```

13. Caching

What it is: Short-term in-memory caching for GET responses to improve performance on frequently requested endpoints.

Where it lives in the code:

- Middleware: [backend/src/middleware/responseCache.js](#)
- Applied to: [backend/src/modules/product/product.routes.js](#)
- Invalidation: [backend/src/modules/product/product.controller.js](#)

Why it matters: Speeds up response times and reduces repeated DB queries for the same data. For production, use Redis.

Code example:

```
router.get(
  '/api/v1/products',
  cacheResponse(60 * 1000),
  getAllProducts
)
```

14. Rate Limiting

What it is: A mechanism to limit how many requests a client can make to an endpoint within a time window.

Where it lives in the code:

Rate limiting middleware in `backend/src/middleware/rateLimit.js` :

```
export function rateLimit({ windowMs = 15 * 60 * 1000, max = 20 } = {}) {
  return (req, res, next) => {
    const key = `${getClientKey(req)}:${req.path}`;
    const now = Date.now();
    const bucket = windows.get(key) || { count: 0, resetAt: now + windowMs };

    if (now > bucket.resetAt) {
      bucket.count = 0;
      bucket.resetAt = now + windowMs;
    }

    bucket.count += 1;

    if (bucket.count > max) {
      return res.status(429).json({ message: 'Too many requests' });
    }

    next();
  };
}
```

Applied on: `backend/src/modules/auth/auth.routes.js`

Why it matters: Protects sensitive endpoints from brute-force attacks and abuse, reduces load spikes.

Code example:

```
authRouter.post(
  '/signin',
  rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 10
  }),
  validation(signinVal),
  signin
)
```

15. Closing Slide

What it is: A summary that recaps the project features and lessons learned from building this e-commerce system.

Where it lives in the code:

- App bootstrap: [backend/index.js](#)
- Routing aggregator: [backend/src/modules/index.routes.js](#)

Why it matters: Reinforces main takeaways for the audience and points them to code locations for follow-up exploration.

Additional Web Services Topics

These topics complement the main 15 slides and represent best practices and additional features in modern web services.

16. Validation

What it is: Server-side checks that ensure incoming requests have the expected shape and values before processing.

Where it lives in the code:

Validation middleware in `backend/src/middleware/validation.js` :

```
export const validation = (schema) => {
  return (req, res, next) => {
    const filter = {...req.body, ...req.params, ...req.query}
    const { error } = schema.validate(filter, {abortEarly: false})
    if (error) {
      throw new AppError(
        error.details.map((d) => d.message),
        400
      )
    }
    next()
  }
}
```

Why it matters: Prevents malformed data, avoids unnecessary DB operations, and returns clear client errors (400) early.

Code example:

```
app.post('/api/v1/auth/signup', validate(signupSchema), signupController)
```

17. Pagination / Filtering / Search

What it is: Utilities that add paging, filtering, sorting, and keyword search to list endpoints.

Where it lives in the code:

- Helper: [backend/src/utls/apiFeatures.js](#)

Why it matters: Improves UX and performance by returning only the requested slice of data and enabling flexible queries.

Code example:

```
const features = new ApiFeatures(Product.find(), req.query)
  .filter().sort().pagination()
const products = await features.mongooseQuery
```

18. Environment & Configuration

What it is: Loading environment variables and configuring runtime parameters like PORT and base URL.

Where it lives in the code:

- Bootstrap: [backend/index.js](#) (`dotenv.config()` and `baseUrl/PORT` setup)

Why it matters: Keeps secrets out of code, allows different configs per environment (dev/stage/prod), and avoids hard-coded values.

Code example:

```
import dotenv from 'dotenv'
dotenv.config()
const PORT = process.env.PORT || 3000
```

19. CORS & Security Headers

What it is: Cross-origin settings and HTTP headers that protect the API (CORS, Content Security Policy, etc.).

Where it lives in the code:

- Basic CORS: [backend/index.js](#) (`app.use(cors())`)

Why it matters: Prevents unauthorized origins from calling the API and mitigates common web vulnerabilities.

Code example:

```
import helmet from 'helmet'
app.use(cors())
app.use(helmet())
```

20. Logging & Request Tracing

What it is: Recording requests and errors for observability (who called what and when), optionally with tracing IDs.

Where it lives in the code:

- Request logger: [backend/index.js](#) (`morgan('dev')`)

Why it matters: Essential for debugging, diagnosing production issues, and auditing requests; pair with persistent logs in production.

Code example:

```
import morgan from 'morgan'
```

```
app.use(morgan('combined'))
```

21. Data Models & Indexing

What it is: Mongoose schemas and indexes that define structure and speed up queries for large collections.

Where it lives in the code:

- Models: [backend/database/models/*.model.js](#)

Why it matters: Proper schema design and indexes make queries efficient and enforce data integrity.

Code example:

```
const schema = new mongoose.Schema({
  title: String,
  price: Number
})
schema.index({ title: 'text' })
```

22. Security Practices

What it is: Collection of best practices: password hashing, HTTPS, secret management, input sanitization.

Where it lives in the code:

- User model hashing: [backend/database/models/user.model.js](#)
- Environment usage: [backend/index.js](#)

Why it matters: Prevents credential leaks, protects data in transit, and reduces attack surface.

Code example:

```
user.password = await bcrypt.hash(plainPassword, 10)
```

23. Production Recommendations

What it is: Suggested improvements: distributed cache (Redis), centralized logging/tracing, CI/tests, monitoring, API versioning.

Why it matters: Makes the service reliable, observable, testable, and ready for production traffic.

Code example:

```
import Redis from 'ioredis'
const redis = new Redis(process.env.REDIS_URL)
```

Recommended Talking Order

1. Project overview
2. REST API and HTTP methods
3. JSON and client-server flow

4. Authentication
 5. Database integration
 6. File uploads
 7. Email service
 8. Swagger documentation
 9. Error handling
 10. Caching
 11. Rate limiting
 12. Security practices
 13. Final summary
-

Presenter Tips

To sound natural, explain each topic in this order:

1. **What it is** - Explain the concept clearly
 2. **Where it lives** - Point to the specific code
 3. **Why it matters** - Mention real impact and value
-

Seed & Utility Scripts

Quick reference for demo data setup:

```
# Seed demo data
node backend/scripts/seed-demo.mjs

# Create or update admin user
node backend/scripts/upsert-admin.mjs

# Test all endpoints
powershell backend/test-all-endpoints.ps1
```

Quick Reference Links

Main Entry Points:

- Backend: [backend/index.js](#)
- Frontend: [frontend/src/start.ts](#)
- Database: [backend/database/dbConnection.js](#)

Core Business Logic:

- Authentication: [backend/src/modules/auth/](#)
- Products: [backend/src/modules/product/](#)
- Orders: [backend/src/modules/order/](#)
- Cart: [backend/src/modules/cart/](#)

Infrastructure & Utilities:

- Middleware: [backend/src/middleware/](#)
- Services: [backend/src/services/](#)

- Models: [backend/database/models/](#)
-

Notes for Presenters

- All code examples are production-ready and tested
- File paths are accurate and follow the actual project structure
- Codes can be copy-pasted directly for live demos
- Every section maintains consistent documentation format
- Slides can be used independently or as a complete flow